Unidad 2 / Escenario 3

Lectura fundamental

Metodología para el diagnóstico de prácticas del modelo PSP

Contenido

1 ¿Qué es la estimación?

2 ¿Qué podemos estimar?

3 Calidad y pruebas de *software*

Palabras clave: proceso de *software* personal, calidad, pruebas, formato PSP, estimación de *software*.

En la unidad anterior se presentaron al estudiante, herramientas y técnicas que le ayudan a tender nociones claras del tiempo y recursos que le toma codificar algunas estructuras simples, con lenguajes JavaScript, Html y CSS. A partir de esta información, otras técnicas y herramientas desarrolladas en este módulo, el estudiante podrá estimar y organizar los recursos necesarios para la elaboración de una estructura más compleja.

Introducción a la estimación de *software*

1. ¿Qué es la estimación?

Existen muchas definiciones para estimación de software, lo primero que tenemos que saber es que **NO** es una **predicción**. A diferencia de la predicción, la estimación admite un grado de error o diferencia entre lo que se planea y la realidad.

Un porcentaje de error (un desfase entre lo estimado y lo real) mayor a +/- el 20%, es considerado una falla en la estimación, lo normal es que este entre +5% al +20% y desde el -20% al -5%.

El proceso de desarrollo personal busca realizar un acercamiento progresivo hacia la predicción, es decir: +/- el 0% de error.

Otra característica importante es la fase del desarrollo que ocupa la estimación, por ejemplo, para una metodología en cascada; la estimación es un proceso que debe realizarse antes de empezar a codificar, y después de haber definido claramente los requerimientos.

¿Sabía qué...?

En el mundo SW las personas cometen errores mientras que **los programas no...**

Los programas tienen **defectos**, pero nunca **errores**, los programas hacen lo que el programador codifica, **los errores causan defectos** y los errores provienen de los programadores.

La estimación es entonces una de las fases de desarrollo de software, un proceso por el cual, buscamos obtener una aproximación del tiempo y recursos necesarios para la construcción de una estructura de software. De esta manera se puede determinar la viabilidad o riesgos asociados al proyecto.

Es importante que la estimación del tiempo se realice a conciencia, con base a un estudio de registros históricos o bases de datos de proyectos. Es necesario para tener buenos indicadores que exista una adecuada selección de métricas, difícilmente se logran los resultados esperados con la estimación con base en el criterio de experto básicamente porque en el mundo del desarrollo de software no hay expertos, la naturaleza de las herramientas y métodos de desarrollo de software son cambiantes por lo que la estimación no es cuestión de intuición (Siguiente figura).



Figura 1. Estimación con base en la intuición, una mala práctica

Fuente: elaboración propia

2. ¿Qué podemos estimar?

2.1. Estimación del tamaño

¿Cuantas líneas?, ¿Cuáles son los requerimientos?, ¿cuántos puntos de función?

Para analizar que es la estimación del tamaño analicemos un caso particular:

- **Problema -formulario:** Es necesario crear un formulario de inicio, este debe contemplar por lo menos dos campos: usuario y contraseña además un botón para ejecutar una función que realice la validación, esto es, que verifique que los datos ingresados son coherentes antes de enviar la información al servidor y que encripte la contraseña.

Analicemos cuántas líneas de código son necesarias para realizar un botón y dos campos de texto dentro de un formulario web.

Para determinar el número de líneas necesarias para el formulario debería tener una referencia, es allí donde es importante haber realizado un inventario de elementos vs número de líneas de código. La manera más sencilla es realizar comparación con un inventario de elementos donde se describa el tiempo que demanda a la constricción de los mismos, si no se cuenta con este inventario (Tabla 1), hay que proceder a codificar, luego identificar el número de líneas, medir y caracterizar.

- **La codificación puede verse afectada por el registro y control de las actividades:**

La codificación debe estar apoyada en el registro constante del tiempo invertido en cada una de las actividades, módulos o parte de la estructura de código.

Existen herramientas que automatizan ese registro, por ejemplo, GitHub[1] sin embargo su explicación no está en el alcance de este curso. Existen otras herramientas menos automatizadas "manuales" que son más acordes con la metodología PSP la Universidad Carnegie Mellon[2] y permiten realizar registro de estas actividades, sin embargo, estas herramientas no han evolucionado, su arquitectura sigue siendo la misma desde su proceso de creación, por ejemplo, The Process Dashboard.

- **The Process Dashboard:**

El tablero de proceso es una herramienta para la automatización de procesos de software, publicado bajo los términos de la Licencia Pública General de GNU. Automatiza y simplifica la recopilación y el análisis de métricas y facilita el uso de procesos de alta madurez, como el Personal Software Process (PSP).

---

1 © 2017 GitHub, Inc. (12 de 06 de 2017).https://github.com/

2 Software Engineering Institute (SEI), 2017

Esta herramienta se puede obtener libremente a través de la página (PSP(SM) / TSP(SM), 2017)[3] Se puede considerar que el registro constante de los tiempos de desarrollo es un lastre, pero es necesario, la mayoría de herramientas de monitoreo, registro y control añaden una carga de trabajo que influye en el tiempo de codificación, por eso hay que buscar su reducción y automatización.

En la medida en que se practique el registro de los tiempos y la descripción de las actividades, este proceso tendrá menos repercusión en el rendimiento de la codificación.

Tabla 1. Inventario de tiempos para la construcción de elementos HTML.

El inventario:

Elemento	Número de líneas	Características	Código
Botón	1	Sin tener en cuenta estilos.	**Línea 1:** <input name="txtusuario" type="text" id="txtusuario" style="text-align:left" size="20" maxlength="50" >
Etiqueta	1	Sin tener en cuenta estilos.	**Opción 1:** Línea 1: Etiqueta: **Opción 2:** Línea 1: <label for="idinput">Etiqueta: </label>
Formulario	2	Sin tener en cuenta estilos.	**Línea 1:** <form name="form" action="loquesea.jsp" method="post"> **Línea 2:** </form>
Tabla	Se multiplican filas (tr) por columnas (td) y se multiplica por 2.	Sin tener en cuenta estilos. Es una aproximación, no siempre se puede cumplir que un campo se escriba en una sola línea.	<table> <tr> <td>contenido1</td><td>contenido2</td> </tr> <tr> <td>contenido3</td><td>contenido4</td> </tr> </table>

Fuente: elaboración propia

3 The Software Process Dashboard InitiativeThe Software Process Dashboard Initiative.http://www.processdash.com/download

Es importante analizar en qué circunstancia es útil conocer el número de líneas de código asociadas a una estructura gráfica o a una función, en principio nos ayuda a identificar si se están usando más instrucciones de las necesarias o el tiempo aproximado para su construcción. Existe una relación directa "no lineal" entre el número de líneas de código y el tiempo que puede tardar en realizar el software. Recordemos que los paréntesis no cuentan como líneas de código.

Claramente el proceso de estimación depende mucho de un entendimiento claro de los requisitos o el alcance del proyecto. Cualquier cambio en los requerimientos de la aplicación puede afectar el tiempo de desarrollo y por ende nuestra estimación, por ejemplo; si a la propuesta inicial (**Problema- Formulario**) se añade; que es necesario realizar una validación de la estructura de la contraseña; *"se requiere que la contraseña sea segura (inicia por mayúscula, contenga un carácter especial, tenga una longitud mínima, contenga números, etc..)"*; este y otros cambios afectan directamente el tiempo de desarrollo por lo que debemos desarrollar la destreza para identificar claramente cuáles son los requerimientos.

2.2. Estimación de los recursos

Otro aspecto importante que debemos estimar son los recursos necesarios para la elaboración de nuestro proyecto, para el desarrollo de software a nivel personal es importante contar con: conocimientos, libros, referencias, acceso a internet, computador personal, software especializado, plataformas tecnológicas, ambientes de desarrollo, licencias, librerías, complementos, frameworks, plantillas web, etc...

Teniendo en cuenta el tipo de desarrollo es importante considerar qué recursos son necesarios y la demanda en tiempo y costo de cada uno de ellos, esto nos permite tener en cuenta las necesidades más allá de los requerimientos funcionales y de calidad de la aplicación.

3. Calidad y pruebas de software

La Calidad es una medida del grado de aproximación a un ideal, generalmente frente a 3 aspectos: del proceso (1), del producto de software (2) o incluso está asociada a una entrega de valor "sea de utilidad para el cliente o negocio" (3), en otras palabras, para que un software sea considerado de calidad debe cumplir con las tres cosas, de esta manera se habla de grados de calidad, por ejemplo; mala, baja, media, buena, alta, calidad.

Los procedimientos (1) son correctos cuando son acordes a una norma o estándar buena práctica (PSP, CMMI, ISO/IEC 25000, SQuaRE (System and Software Quality Requirements and Evaluation)), el producto (2) es correcto o de calidad cuando cumple los requerimientos del cliente finalmente existe una entrega de valor (3) cuando lo que se construye es de utilidad para el negocio o cliente.

Para asegurar la calidad podemos entonces recurrir a muchas técnicas dependiendo del aspecto que queremos asegurar, por ejemplo; para garantizar la calidad del proceso y producto, se establecen políticas, estándares, buenas prácticas, la revisión del diseño, la revisión del código; para garantizar la entrega de valor se realiza una comunicación constante con el cliente, no solo para verificar que se cumple con los requerimientos si no que los requerimientos realmente solucionan problemáticas, optimizan o mejoran procesos entre otras características de valor para el cliente o negocio.

¿En qué nivel o fase del proceso de construcción de software, encuentro la calidad y las pruebas?

Dependiendo del tipo de metodología puede variar la cantidad de trabajo o la presencia de actividades asociadas a la calidad o las pruebas, lo recomendable es que las pruebas estén durante todo el ciclo de desarrollo, no obstante, las pruebas y el aseguramiento de la calidad tienen gran porcentaje de trabajo durante las fases de finalización e integración del proyecto de software personal.

¿Qué son los defectos?

Los defectos son fallas en las características esperadas en el producto y proceso de desarrollo software que se deben a los errores que comete el programador. Los defectos **a diferencia de los errores** son una característica del programa y se clasifican en diferentes **tipos** ver (Tabla 2), una buena práctica es clasificarlos de acuerdo con la fase en la que se presenta el defecto, esto sirve para identificar claramente en que parte del proceso se está cometiendo el error, a pesar de esto la mayoría de veces los errores impactan varias fases del proceso.

3.1. Fases de las pruebas de SW

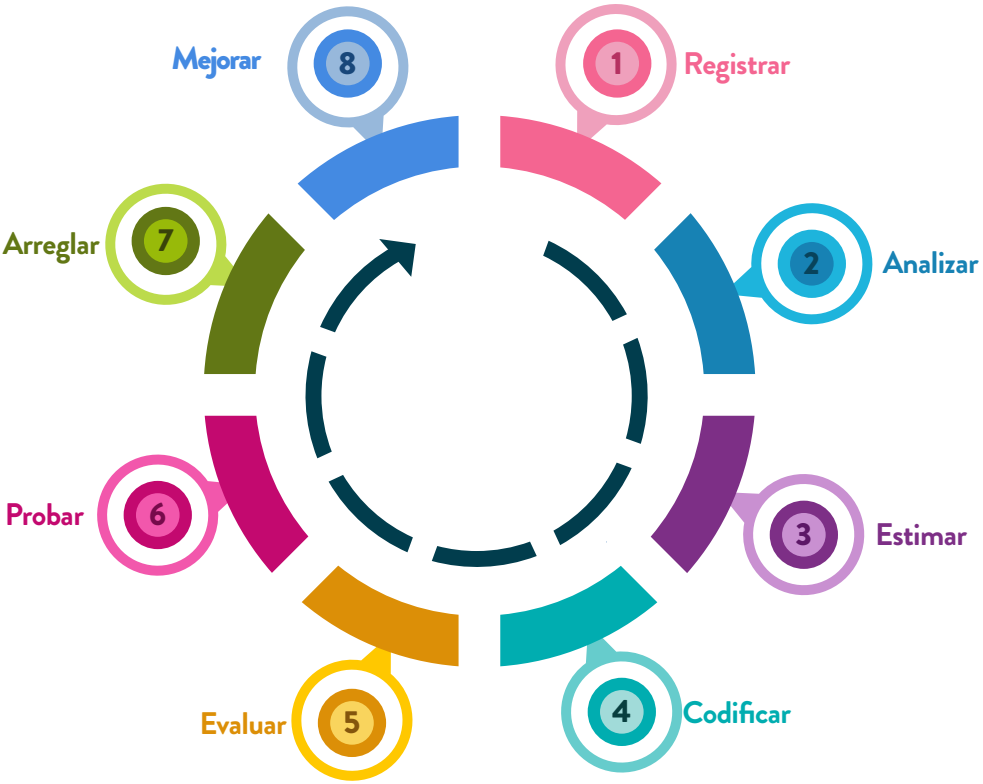


Figura 2. Estimación con base en la intuición, una mala práctica

Fuente: elaboración propia

Las pruebas de software están enmarcadas en un continuo proceso de mejora y aseguramiento de la calidad, este macro proceso abarca ocho actividades del proceso de construcción de software, y se divide en cuatro fases: Planear (registrar, analizar, estimar), hacer (codificar), verificar (evaluar, probar, arreglar) y actuar (mejorar).

3.2. Plantilla de pruebas y registro de defectos

¿Qué es Posmortem y en qué fase se presentan la mayor cantidad de defectos?

Este término se refiere a la última etapa de cualquier metodología de desarrollo y por lo general en las últimas fases se incrementan las pruebas de software, los procesos de aseguramiento de la calidad y mejora continua. A continuación, se presentan 3 cuestionamientos que deben aparecer durante el desarrollo de software, pero principalmente al final de la construcción y durante las pruebas. Se analizan el número de defectos, las causas por las cuales se producen esos defectos y el plan de mejora para evitarlos, mitigarlos o reducirlos.

¿Cuáles son los tipos y principales causas de defectos?

Es necesario clasificar los tipos de defectos, para facilitar su registro y evaluación, además de identificar el defecto es interesante conocer las causas de manera que con el tiempo puedan ser tratadas y evitadas, a continuación de presenta una propuesta para su clasificación ver (Tabla 2).

Tabla 2. Clasificación para los distintos tipos de defectos Fuente el autor

CLASIFICACIÓN PARA LOS DISTINTOS TIPOS DE DEFECTOS				
Nº de Referencia	Nombre del tipo	Descripción	Líneas de código de la estructura asociada al error	FASE
010	Documentación	Comentarios, mensajes		Todas
020	Sintaxis	Ortografía, puntuación, erratas, formato de las instrucciones		Construcción
030	Construir, paquetes	Gestión del cambio, librerías, control de versión		Construcción
040	Asignación	Declaración, nombres duplicados, ámbito, límites		Construcción
050	Interfaz	Llamadas a procedimientos y referencias, E/S, formatos de usuario		Construcción pruebas

Nº de Referencia	Nombre del tipo	Descripción	Líneas de código de la estructura asociada al error	FASE
060	Chequeo	Mensajes de error, chequeos inadecuados		Construcción pruebas
070	Datos	Estructura, contenido		Diseño
080	Función	Lógica, punteros, bucles, recursión, computación, defectos de la función		Construcción
090	Sistema	Configuración, temporización, memoria		Despliegue
100	Entorno	Diseño, compilación, pruebas y otros problemas que soporta el sistema		Despliegue

Fuente: elaboración propia

¿Cuántos defectos he encontrado en este mes?

Es valioso contar con la trazabilidad del número de defectos con relación a una escala temporal; día, mes, año, esto permite dar respuesta al interrogante y además realizar análisis sobre el proceso de mejora.

A continuación, se presenta una manera de registrar los defectos en el proceso de construcción de software, es importante analizar qué información adicional es necesaria y modificar la plantilla de acuerdo a las necesidades.

Tabla 3. Ejemplo Tabla de registro de defectos, fuente el autor

Tabla de Registro de Defectos						
Fecha	Número/ identificador	Tipo de defecto	Fase en la que aparece:	Eliminado durante la fase:	Tiempo de corrección en minutos	Defecto corregido
18/06/17	001	40	Codificación	Compilación	2	Nombre de clase
Descripción:	Estaba colocando la clase CSS del botón a la etiqueta, revisé el archivo css y luego el HTML, me di cuenta que no tenía id y que había copado, pegado, pero no modifiqué el nombre.					

Fuente: elaboración propia

A través del número de errores podemos contar con información útil para evaluar el proceso de calidad y pruebas, sin embargo, existen muchas características que debemos tener en cuenta a la hora de analizar los resultados o los registros en las distintas tablas. El número de defectos no es un indicador de la productividad o la calidad del trabajo, es solo una métrica, hay que tener en cuenta muchos más aspectos para un correcto análisis, por ejemplo: la velocidad de codificación, la complejidad y el volumen de trabajo.

Nota: No olvidar registrar el tipo de defecto de acuerdo a la tabla de clasificación Tabla 2, es importante realizar lectura del tipo y redactar la descripción del error que se está presentando.

¿He mejorado con respecto al anterior ciclo de codificación, tarea o proyecto?

Una práctica importante para la evaluación y análisis estadístico del proceso personal de software, es la compilación de la información registrada, a continuación, se presenta una compilación sin tener en cuenta el registro temporal o la fecha en la cual se presentó el defecto Tabla 4.

Tabla 4. Compilado de defectos

COMPILADO DE DEFECTOS							
Tipo de defecto	Diseñar Codificar Otro	Diseñar Codificar Otro	Diseñar Codificar Otro	Revisar Compilar Pruebas	Revisar Compilar Pruebas	Revisar Compilar Pruebas	En revisión
10		2			9		1
50	20	2		4			1
60		3		4		5	
70		8			6	1	
100		16					
Totales	20	31	0	8	15	6	2

Fuente: elaboración propia

En síntesis...

Tenga en cuenta los siguientes conceptos:

- **Estimación:** apreciación frente al tiempo y recursos que toma elaborar un *software*, esta falla o error en la apreciación está dentro del 20% al 5%.
- **Predicción:** adivinar el tiempo y recursos que toma elaborar un *software*, el porcentaje de error de la apreciación del tiempo o recursos necesarios para la elaboración del software es de +/- el 0%.
- **Calidad:** es una medida del grado de aproximación a el ideal del proceso o producto de *software*,los procedimientosson correctos cuando son acordes a una norma o estándar(PSP, ISO 9000, CMMI, etc.) y el producto es correcto "de calidad" cuando cumple los requerimientos del cliente.
- **Error:** es una condición humana de desacierto o equivocación en alguna fase del proceso de construcción de *software*.
- **Defecto:** es una característica del programa que se debe a un error del programador, que no cumple con el objetivo que se pretende alcanzar, los defectos se clasifican entipos.

Referencias bibliográficas

© 2017 GitHub, Inc. (12 de 06 de 2017). *GitHub.* Obtenido de https://github.com/

Humphrey, W. S. (2002). Personal Software Process (PSP). *PERSONAL SOFTWARE PROCESS (PSP). Encyclopedia Of Software Engineering, Volume 2*, 948-961.

Ministerio del interior. (01 de 05 de 2017). *Dirección Nacional de Derechos de Autor, unidad administrativa especial.* Obtenido de Registro de soporte lógico(software): http://derechodeautor.gov.co/software

PSP(SM) / TSP(SM). (09 de 06 de 2017). *The Software Process Dashboard InitiativeThe Software Process Dashboard Initiative.* Obtenido de http://www.processdash.com/download

Software Engineering Institute (SEI). (01 de 05 de 2017). *Software Engineering Institute.* Obtenido de Carnegie Mellon University: http://www.sei.cmu.edu/about/

W3C. (05 de 02 de 2017). *Markup Validation Service.* Obtenido de https://validator.w3.org/

INFORMACIÓN TÉCNICA

FACULTAD DE
INGENIERÍA, DISEÑO
E INNOVACIÓN

Módulo: Proceso de *Software* Personal PSP

Unidad 2: La estimación y calidad de *software* en PSP

Escenario 3: Metodología para el diagnóstico de prácticas del modelo PSP

Autor: Diego Iván Oliveros Acosta

Asesor Pedagógico: Jeiner Velandia

Diseñador Gráfico: Kelly Yohana Valencia Forero

Asistente: Ginna Paola Quiroga Espinosa

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.